Lecture 10: Quarto

Table of contents

1 Overview

This week, you will be introduced to Quarto. Quarto is a powerful editor which incorporates plain text and computer code together, and it can easily be rendered into different output formats (html, pdf, .doc, etc.). Quarto is especially useful for writing reports, summarizing/analyzing data, incorporating plots, etc. You will follow some self-guided reading and will submit a Quarto file (.qmd) as well as the rendered HTML document (.html) for Homework 10 (Due at the end of next week).

2 Setup

2.0.1 Install packages

- Run the following code in your console before opening a quarto document

```
install.packages("here")
install.packages("rmarkdown")
```

2.1 Open a Quarto document

- New file dropdown icon in top right of Rstudio
- New Quarto Document
- Title: My First Quarto
- Select HTML
- Engine = knitr
- Select "Use Visual Editor"

3 Lecture Outline

3.1 Get to know Quarto

There are three main sections

1. An (optional) **YAML header** surrounded by `---`s.
2. **Chunks** of R code surrounded by ```` ``` ````.
3. Text mixed with simple text formatting like `# heading` and `_italics_`.

We will discuss these in order, but first a note on the editors

- You can use the "visual" editor or the "source" editor
  - Visual is similar to what you have used in other programs like Word, Google Docs, etc.
    - This is called "what you see is what you get" (*wysiwyg*) text editors
    - when you highlight text and click the 'boldface' icon, the text appears bold on your screen
    - Likewise you can change the heading level, change the font to represent code, etc.
  - Source is a simple text editor

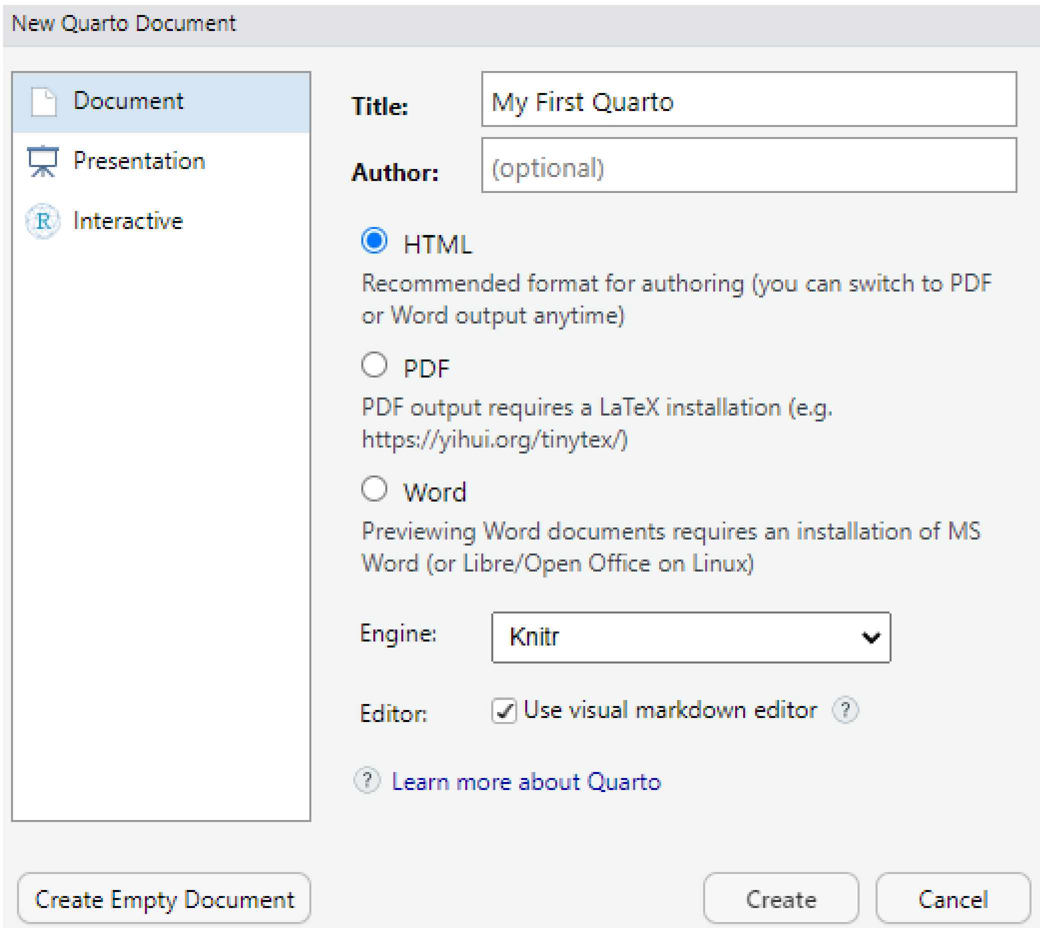

Figure 1: Screenshot showing details for pop-up window for your first quarto document

    - The biggest difference is that formatting occurs in the document itself rather than behind the scenes
    - Switch to the source editor
    - Notice the `##` for the headers, and the `*` around the word render to make it bold
  - The visual editor is helpful, but it's important to also know the source editor
    - The visual editor can be glitchy, and sometimes the only way to fix it is with the source editor

- Try and make the bullet-point error in class as an example
  - Start a bulleted list
  - Add some subpoints in between
- It get's "stuck" in bullet-mode
- Go to source editor and delete bullets, add a new line to get out of bullet mode
- Hanging bullets sometimes turn into code
- delete the code chunk designation: \` \` \`

3.2 Why use Quarto

R Markdown has many advantages compared to creating reports in Word or GoogleDocs. These advantages include:

1. **Versatility** - Want to convert a Word document into pdf? That's not too hard. But pdf to Word? That's a pain (or at least it used to be). PDF to HTML? Maybe you know how to do that but I don't. With Quarto, we can change between these formats (or produce multiple outputs at once) with a few lines of code). You can even convert them into pretty nice slide shows.
2. **Embed code in text** - After running an analysis, how do you get your results into Word? Type them by hand? Copy-and-paste? Both are a pain and error prone. Rerun your analysis using new data? Oops, now you have to copy and paste those new results and figures. With Quarto, we embed code directly into the text so results and figures get added to our reports automatically. That means no copying and pasting and updating reports as new results come in is pain free. I cannot emphasize enough how convenient this is. This will certainly save you time in the future.

3. **Annotate your code** - Using the `#` is great for adding small annotations to your R scripts and you should definitely get in the habitat of doing that. But sometimes you need to add a lot of details to help other users (or your future self) make sense of complex code. For instance, in this class *interpreting* the results of statistical analyses can be detailed and complex. Writing this in Quarto will be much easier, and reading the html document will be easier for me to grade! Quarto allows you to create documents with as much text and formatting as you need, along with the code.
4. **Version control** - Tired of saving "manuscript_v1.doc", "manuscript_v2.doc", "manuscript_final.doc", "manuscript_final_v2.doc"? Then version control is for you. We won't go into the specifics here but Quarto allows you to seamlessly use version control systems like git and Github to document changes to your reports.
5. **Edit as text files** - Quarto files are most easily created and edited within RStudio but you don't have to do it that way. They can be opened and edited in base R and even using text editors. This means you can create and edit them on any platform (Windows, Mac, Linux) using free software that is already installed on the computer
6. **Focus on text, not formatting** - Do you spend a lot of time tweaking the formatting of your Word document rather than writing? Quarto allows you to separate the writing process from the formatting process, which allows you to focus on the former without worrying about that later (in theory at least). Plus there are lots of templates you can use to ensure that the formatting is taken care without you having to do anything special!

3.3 YAML Header

- At the top of your `.qmd` file, you should see several lines of code in between three blue dashes

```
---
title: "My First Quarto"
format: html
editor: visual
---
```

- This is called the "YAML header"
- it's where we can control a lot of the major formatting options for our documents.
- to change the output to pdf, just switch `format: html` to `format: pdf`
  - **note**, you may need to install a Latex distribution to knit to pdf. If you get an error message at this step
  - Try running `install.packages("tinytex")`
  - If this doesn't work, try clicking in your terminal (tabl in console pane) and type:

    - `quarto install tinytex`

- There's lots of formatting options you can include in the YAML header, most of which are beyond the scope of this course.
- For this class we're going to stick with HTML output, but we will include table of contents by including the following code:

```
toc: true
toc-depth: 2
number-sections: true
```

3.3.1 Libraries

```r
library(tidyverse)
```

3.3.2 Data

```r
rxp <- read.csv("data/RxP.csv",
                stringsAsFactors = TRUE)
```

4 Summary